

Microatividade1: Descrever a utilização das estruturas de condição if e else em Python

Foi criada uma variável chamada temperatura e atribuída a ela o valor 29.

Foi criada uma verificação, utilizando a condição if, para checar se o valor da variável temperatura é menor que 30.

Caso positivo, na tela será imprimido o texto 'A temperatura hoje está amena'.

Caso contrário, e utilizando a condição else, será imprimido na tela o texto 'Hoje está fazendo calor'.

```
estruturas_condicao1.py >...  
1  temperatura = 31  
2  # cria condição que verifica se a temperatura é menor  
3  if temperatura < 30:  
4      print("A temperatura hoje está amena")  
5  else:  
6      print("Hoje está fazendo calor")  
7
```

Resultado:

```
PS C:\Users\miguel.evangelista\microatividades> & C:/Users/miguel.evangelista/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/miguel.evangelista/microatividades/estruturas_condicao1.py  
Hoje está fazendo calor
```

Microatividade 2: Descrever a utilização da estrutura de condição else if (elif) em Python

Foi criada uma variável "tempoExperiencia" que receberá o valor do tempo de experiência do usuário

Foi verificado se o usuário possui 2 ou menos anos de experiência, caso sim será retornado 'Nível de conhecimento júnior.'

Foi verificado se o usuário possui mais de 2 anos de experiência e menos de 5 anos de experiência, caso sim será retornado 'Nível de conhecimento pleno.'

Por fim foi atribuído a condição else, se o tempo de experiência não cumprir os requisitos de verificação anteriores será retornado 'Nível de conhecimento sênior.'

```
estruturas_condicao2.py > ...
1 tempoExperiencia = 3
2 # verifica se o tempo de experiencia é menor que 2
3 if tempoExperiencia <= 2:
4     print("Nível de conhecimento júnior.")
5 # verifica se o tempo de experiencia é maior que 2 men
6 elif tempoExperiencia > 2 and tempoExperiencia < 5:
7     print("Nível de conhecimento pleno.")
8 # se o tempoExperiencia não se encaixar em nenhuma das
9 else:
10    print("Nível de conhecimento sênior.")
```

Resultado:

```
PS C:\Users\miguel.evangelista\microatividades> & C:/Users/miguel.evangelista/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/miguel.evangelista/microatividades/estruturas_condicao2.py
Nível de conhecimento pleno.
PS C:\Users\miguel.evangelista\microatividades>
```

Microatividade 3: Descrever a utilização da estrutura de repetição while em Python

O programa inicializa uma chamada variável entrada_idade como uma "").

Em seguida, entra em um loop while, Durante o loop:

O usuário é solicitado a digitar um número ou "0" para sair.

Caso o número digitado não seja "0", ele será exibido no console.

Quando o usuário digita "0", o programa sai do loop e imprime a mensagem "Programa encerrado."

```

estruturas_repeticao1.py > ...
1  entrada_idade = ''
2
3  # Loop enquanto a entrada não for 0
4  while str(entrada_idade) != '0' :
5  # Solicita ao usuário para digitar um número ou 0 pa
6  |     entrada_idade = input('Digite um número qualquer
7  | # Verifica se o número é 0 para evitar imprimir após
8  |     if entrada_idade != '0':
9  |         print(f"Número digitado: {entrada_idade}")
10
11 print("Programa encerrado.")
12

```

Resultado:

```

PS C:\Users\miguel.evangelista\microatividades> & C:/Users/miguel.evangelista/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/miguel.evangelista/microatividades/estruturas_repeticao1.py
Digite um número qualquer ou 0 para sair: 50
Número digitado: 50
Digite um número qualquer ou 0 para sair: 0
Programa encerrado.
PS C:\Users\miguel.evangelista\microatividades>

```

Microatividade 4: Descrever a utilização da estrutura de repetição for em Python

O programa inicializa uma string chamada texto com o valor 'Olá, laço for.'.

Cria uma lista vazia chamada item.

Usa um laço for para iterar sobre cada caractere da string texto e adiciona esses caracteres à lista item.

Após o loop, exibe no console o conteúdo da lista item, que conterà os caracteres individuais da string.

Um laço for é usado com a função range(10), que gera números de 0 a 9.

Para cada número gerado, o programa imprime a mensagem: "Número do intervalo: {número}", substituindo {número} pelo valor atual do número.

```

estruturas_repeticao2.py > ...
1  texto = 'Olá, laço for.'
2
3  item = []
4
5  for caractere in texto:
6      item += caractere
7
8  print(item)
9
10 for numero in range (10):
11     print('Número do intervalo: ' + str(numero) )
12

```

Resultado:

```

PS C:\Users\miguel.evangelista\microatividades> & C:/Users/miguel.evangelista/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/miguel.evangelista/microatividades/estruturas_repeticao2.py
['O', 'l', 'á', ',', ' ', 'l', 'a', 'ço', ' ', 'f', 'o', 'r', '.']
Número do intervalo: 0
Número do intervalo: 1
Número do intervalo: 2
Número do intervalo: 3
Número do intervalo: 4
Número do intervalo: 5
Número do intervalo: 6
Número do intervalo: 7
Número do intervalo: 8
Número do intervalo: 9
PS C:\Users\miguel.evangelista\microatividades>

```

Microatividade 5: Descrever a utilização de funções em Python

A função é definida com o nome “imprimir_variavel”.

Dentro da função, uma variável chamada “texto” é declarada e recebe o valor da string “Olá, funções em Python”.

O conteúdo da variável “texto” é exibido no console utilizando a função “print”.

A função “imprimir_variavel” é chamada explicitamente, o que executa o código contido em seu bloco.

```

funcoes1.py > ...
1  def imprimir_variavel():
2      texto = "Olá, funções em Python"
3
4      print(texto)
5
6  imprimir_variavel()

```

Resultado:

```

sta/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/miguel.eva
ngelista/microatividades/funcoes1.py
Olá, funções em Python
PS C:\Users\miguel.evangelista\microatividades>

```

Microatividade 6: Descrever a utilização argumentos de funções no Python

A função "loginUsuario" recebe um parâmetro chamado perfil.

Verifica se o valor do parâmetro, convertido para letras minúsculas (usando o método .lower()), é igual a "admin".

Se for igual, exibe a mensagem "Bem-vindo, Administrador".

Caso contrário, exibe a mensagem "Bem-vindo, Usuário".

A função é chamada com o argumento "Admin", o que dispara a verificação e imprime a mensagem correspondente.

```

funcoes2.py > ...
1  def loginUsuario(perfil):
2      if perfil.lower() == "admin":
3          print("Bem-vindo, Administrador")
4      else:
5          print("Bem-vindo, Usuário")
6
7  loginUsuario("Admin")

```

Resultado:

```
PS C:\Users\miguel.evangelista\microatividades> & C:/Users/miguel.evangelista/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/miguel.evangelista/microatividades/funcoes2.py
Bem-vindo, Administrador
PS C:\Users\miguel.evangelista\microatividades> |
```

Trabalho Prático

Foram criadas as funções iniciais para o projeto, `adicao(num1, num2)`: calcula e exibe a soma de dois números.

`subtracao(num1, num2)`: Calcula e exibe a diferença entre dois números.

`multiplicacao(num1, num2)`: Calcula e exibe o produto de dois números.

`divisao(num1, num2)`: Verifica se algum dos números é zero e exibe uma mensagem de erro. Caso contrário, calcula e exibe o quociente dos dois números.

```
calculadora_v2.py > ...
1  valor = ''
2  # cria funcoes de adicao subtracao multiplicacao e divisao
3  def adicao(num1, num2):
4      soma = num1 + num2
5      print(soma)
6
7  def subtracao(num1, num2):
8      sub = num1 - num2
9      print(sub)
10
11 def multiplicacao(num1, num2):
12     mult = num1 * num2
13     print(mult)
14
15 def divisao(num1, num2):
16     if num1 or num2 == 0:
17         print("Não foi possível realizar a divisão por 0")
18     else:
19         div = num1 / num2
20         print(div)
```

Posteriormente foi criada a função `calculadora` que recebe dois números (`num1` e `num2`) e uma operação (`operacao`) como parâmetros.

Verifica o tipo de operação solicitada e realiza o cálculo correspondente:

Adição: Reconhece `"adicao"`, `"adição"` ou `"+"`.

Subtração: Reconhece `"subtracao"`, `"subtração"` ou `"-"`.

Multiplicação: Reconhece `"multiplicacao"`, `"multiplicação"` ou `"*"`.

Divisão: Reconhece "divisao", "divisão" ou "/". Caso algum número seja zero, exibe uma mensagem de erro.

Retorna o resultado da operação ou uma mensagem de erro caso a operação seja inválida.

```
def calculadora(num1, num2, operacao):
    if operacao in ["adicao", "adição", "+"]:
        resultado = num1 + num2
    elif operacao in ["subtracao", "subtração", "-"]:
        resultado = num1 - num2
    elif operacao in ["multiplicacao", "multiplicação", "*"]:
        resultado = num1 * num2
    elif operacao in ["divisao", "divisão", "/"]:
        if num1 or num2 != 0:
            resultado = num1 / num2
        else:
            print("Não foi possível realizar a divisão por 0")
    else:
        resultado = "Operação inválida."

    return resultado
```

Por fim, foi criado um loop que enquanto o valor da variável “saída” for diferente de “S” será chamado o método “Try” criando os inputs que solicitam ao usuário os seguintes dados: primeiro (num1) e o segundo número (num2) e a operação desejada (adição, subtração, multiplicação, divisão ou seus símbolos +, -, *, /). Foi chamada e armazenada em uma variável “resultado” a função calculadora passando num1, num2 que foram os inputs criados e operação como parâmetros de execução, se tudo ocorrer corretamente será exibido o valor do cálculo conforme a operação solicitada pelo usuário, caso ocorra algum erro será impresso pelo método catch: "Erro: Insira valores válidos.". Ao fim será atribuído a variável saída um input com o valor "Deseja sair? Digite S para sair ou N para continuar: ", caso a resposta seja S o programa deverá ser encerrado e caso a resposta seja N o programa iniciará o loop novamente solicitando ao usuário que informe através dos inputs o “num1”, “num2” e a operação para um novo cálculo

```
saida = "N"
while saida.upper() != "S":
    try:
        # cria os inputs solicitando os valores e retorna o resultado da operacao desejada
        num1 = float(input("Digite o primeiro número: "))
        num2 = float(input("Digite o segundo número: "))
        operacao = input("Digite a operação desejada (adição, subtração, multiplicação, divisão ou seus símbolos +, -, *, /): ")
        resultado = calculadora(num1, num2, operacao)
        print(f"Resultado da operação: {resultado}")
    except ValueError:
        print("Erro: Insira valores válidos.")
        continue
    saida = input("Deseja sair? Digite S para sair ou N para continuar: ")
```

Resultado:

```
PS C:\Users\miguel.evangelista\microatividades> & C:/Users/miguel.evangelista/AppData/Local/Microsoft/Windows/SoftwareDistribution/Download/Python27/python.exe C:/Users/miguel.evangelista/AppData/Local/Microsoft/Windows/SoftwareDistribution/Download/Python27/python.exe a v2.py
a v2.py
Digite o primeiro número: 15
Digite o segundo número: 5
Digite a operação desejada (adição, subtração, multiplicação, divisão ou seus símbolos +, -, *, /): +
Resultado da operação: 20.0
Deseja sair? Digite S para sair ou N para continuar: n
Digite o primeiro número: 30
Digite o segundo número: 5
Digite a operação desejada (adição, subtração, multiplicação, divisão ou seus símbolos +, -, *, /): *
Resultado da operação: 150.0
Deseja sair? Digite S para sair ou N para continuar: s
PS C:\Users\miguel.evangelista\microatividades>
```